





Professional
Master's Degree
in Advanced
Engineering of
Robotic Systems
and Artificial
Intelligence
(IASRIA)



Integration of an Automated Defense Module **NOMAD SAMURAI**



Ramzi Abidi

Committee

Mr.Riadh Salah Chaouachi

Ms Chiraz Ben Jabeur

Mr.Foued El Kamel

Mr. Wassim Arfa

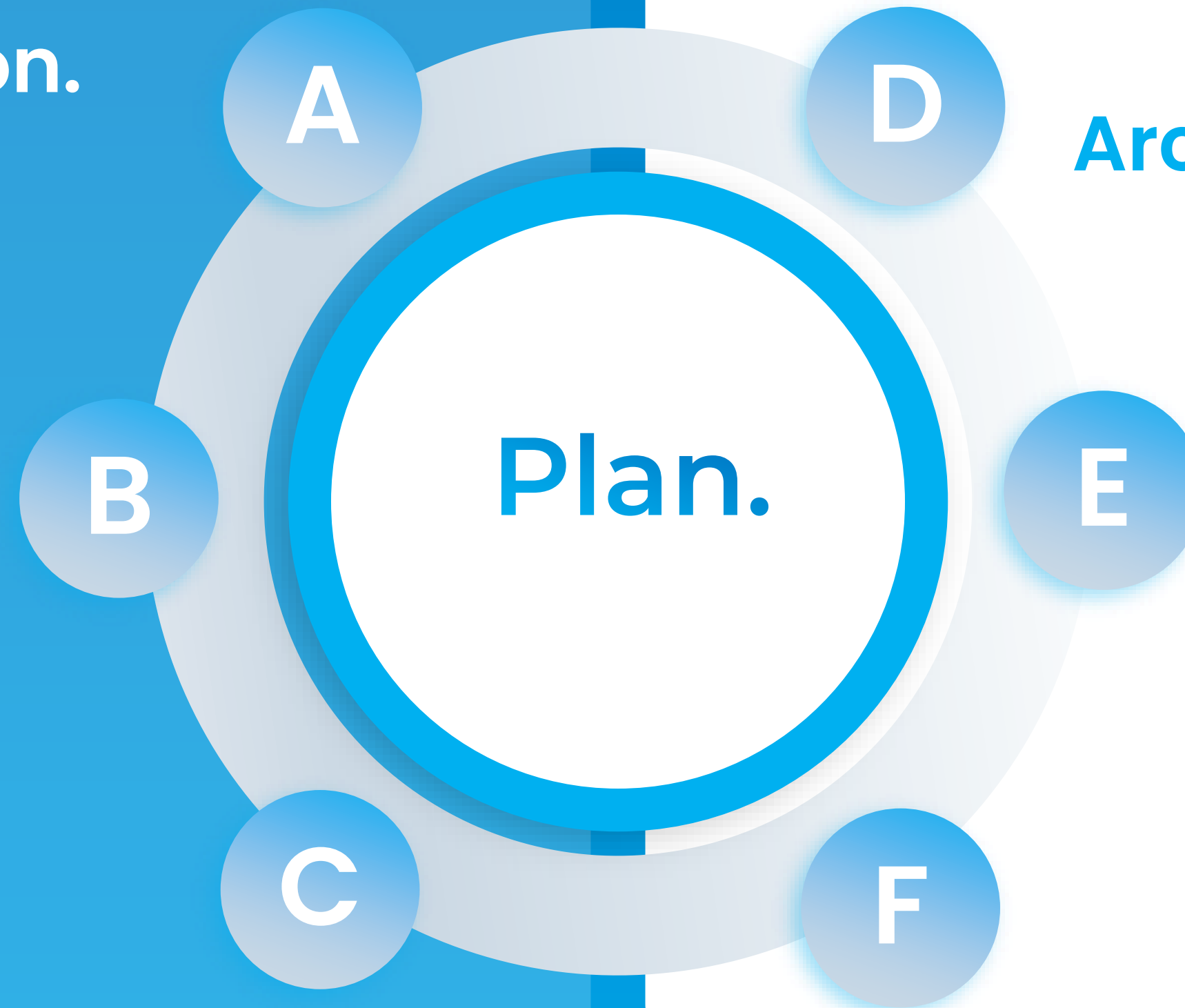


Plan.

Introduction.

Problem.

Solution.



**H/s
Architecture.**

**Implementation
& Simulation.**

Conclusion.

A

Presentation of the company

AVIONAV is a Tunisian S.A.R.L. company based in Borjine, Sousse, that specializes in designing and building light aircraft. Originally founded in Italy in 1990, the company relocated to Tunisia in 2007 and, a decade later in 2017, was acquired by Foued El Kamel, a computer engineer and co-founder of Oxygene Aeronautics, alongside his brother.



B

C

D

A

B

C

D

E



Problem Statement

A

B

C

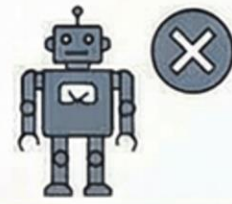
D

E

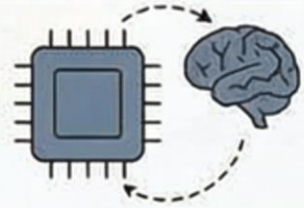
Problem Statement

Problem Statement: Limitations & Critical Needs in Tunisia

Current Robotic System Limitations



Lack of Modularity



No Real-time AI Integration



**Absence of Simulation -
based Validation**

Critical National Needs



Border Surveillance



Demining Operations



**Infrastructure
Inspection**

Urgency for the NOMAD Solution : Development of an intelligent modular robotic platform (NOMAD SAMURAI) is critical to address these gaps.

B

C

D

E

F

Benchmark and Solution

Unmanned Ground Vehicles (UGVs) have become integral to modern defense due to their modular adaptability. Their **design** supports diverse operational conditions by integrating multiple defense modules onto a single platform. This study provides a detailed analysis of leading active rovers and their efficiency in integrating these mechanisms.



B

C

D

E

F

BENCHMARKING LEADING MILITARY UGVs

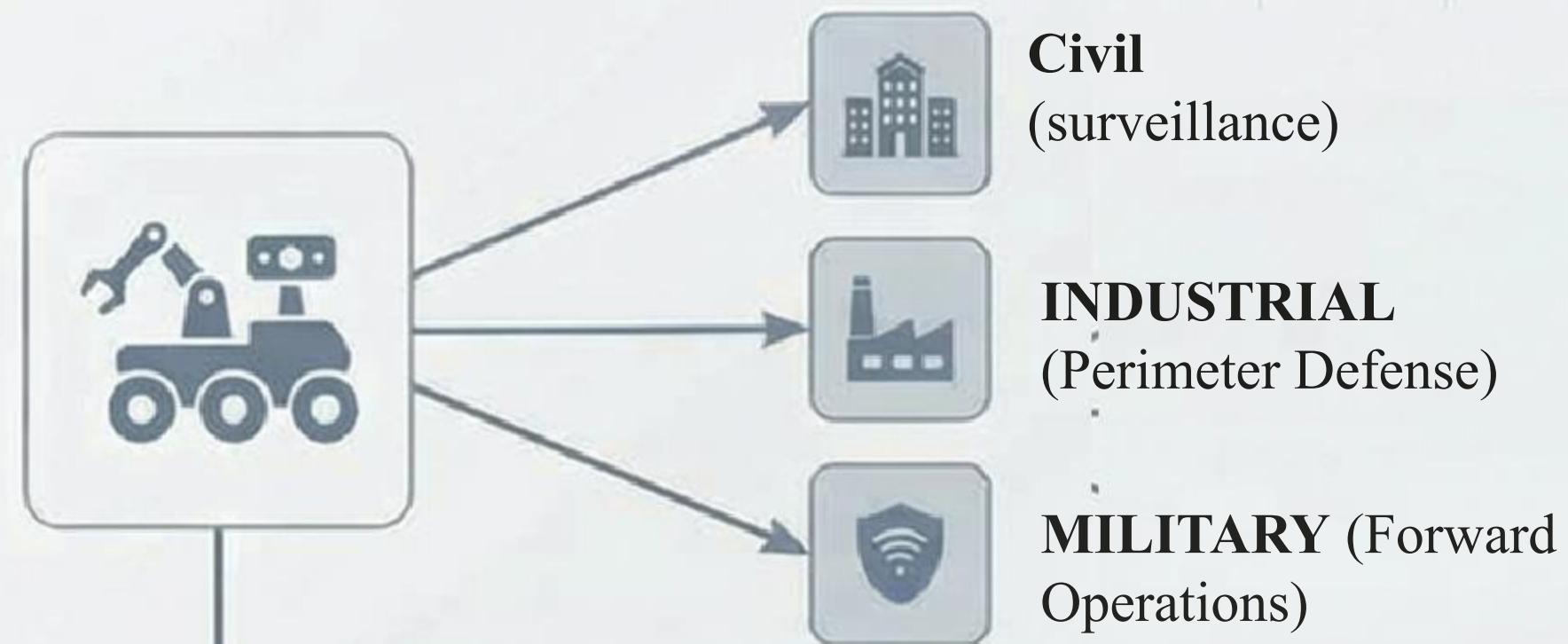
MAARS , THeMIS , Ripsaw and TypeX : Modular Armament , Reconnaissance and Adaptability Analysis

	MAARS	THeMIS	Ripsaw	TypeX
MODULAR ARMAMENT	 Light/Medium Caliber, Guided Missiles	 Light/Medium Caliber, Guided Missiles	 Light/Medium Caliber, Guided Missiles	 Light/Medium Caliber, Guided Missiles
RECONNAISSANCE PAYLOADS	 Electro-Optical/Infrared Sensors, Radar	 Electro-Optical/Infrared Sensors, Radar	 Electro-Optical/Infrared Sensors, Radar	 Electro-Optical/Infrared Sensors, Radar
MISSION ADAPTABILITY	 Logistics,Combat Support, Surveillance	 Logistics,Combat Support, Surveillance	 Logistics, Combat Support, Surveillance	 Logistics,Combat Support, Surveillance
SAMURAI TARGET BENCHMARKS	 Exceeds current Autonomy & Sensor Integration in a lighter, AI-first form factor  SAMURAI VISION : Leveraging AI - First design for superior , lighter and more adaptable defense modules .			



CRITISIM OF THE EXISTING

**ADAPTABILITY
ACROSS MISSIONS &
KEY CAPABILITIES**



Project Purpose



PROJECT MISSION:

To develop an intelligent system that automates high-risk surveillance and response tasks across varied terrains, reducing human exposure and enhancing situational awareness.

CORE SYSTEM OBJECTIVES



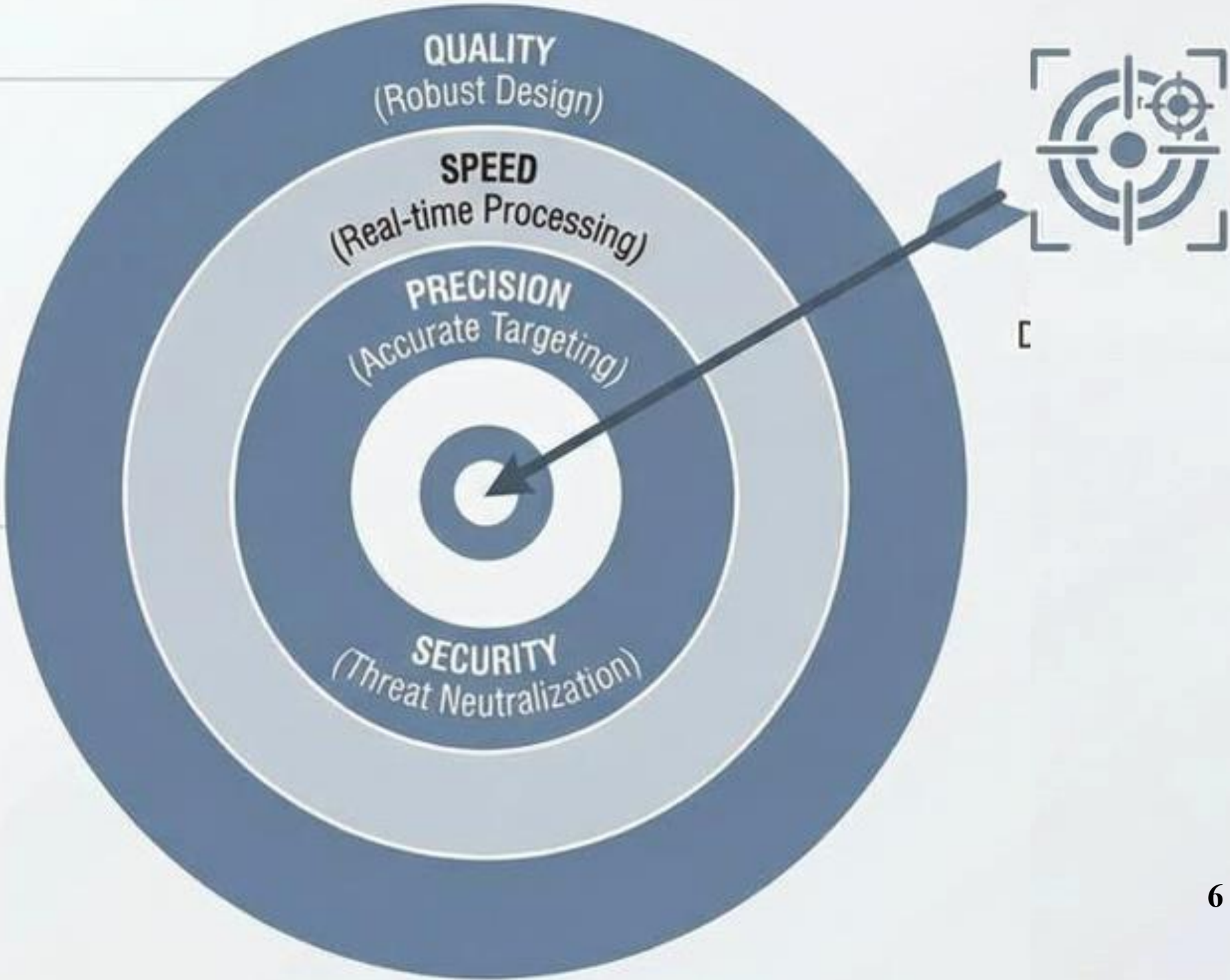
AUTOMATED
Detection



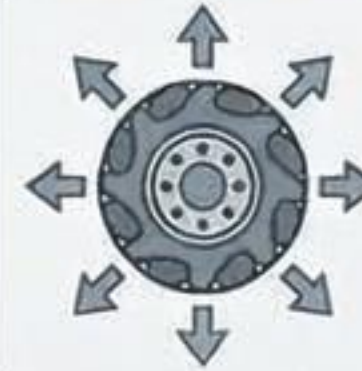
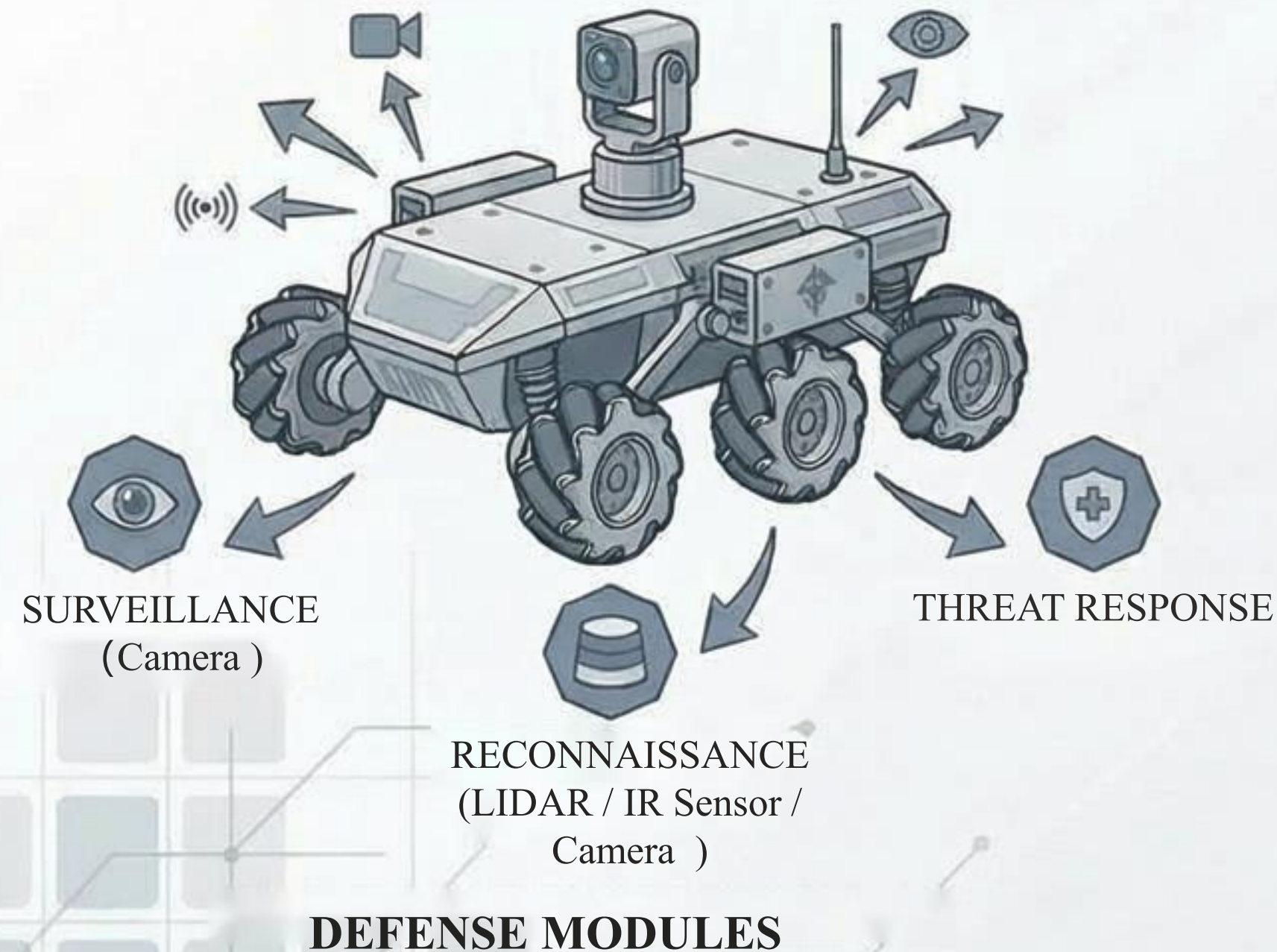
HUMAN AID



TERRAIN ADAPTABILITY



MODULAR ARCHITECTURE



OMNIDIRECTIONAL MOBILITY

- Holonomic Movement
- Precision Maneuvering
- Dynamic Stability



SENSOR FUSION FRAMEWORK

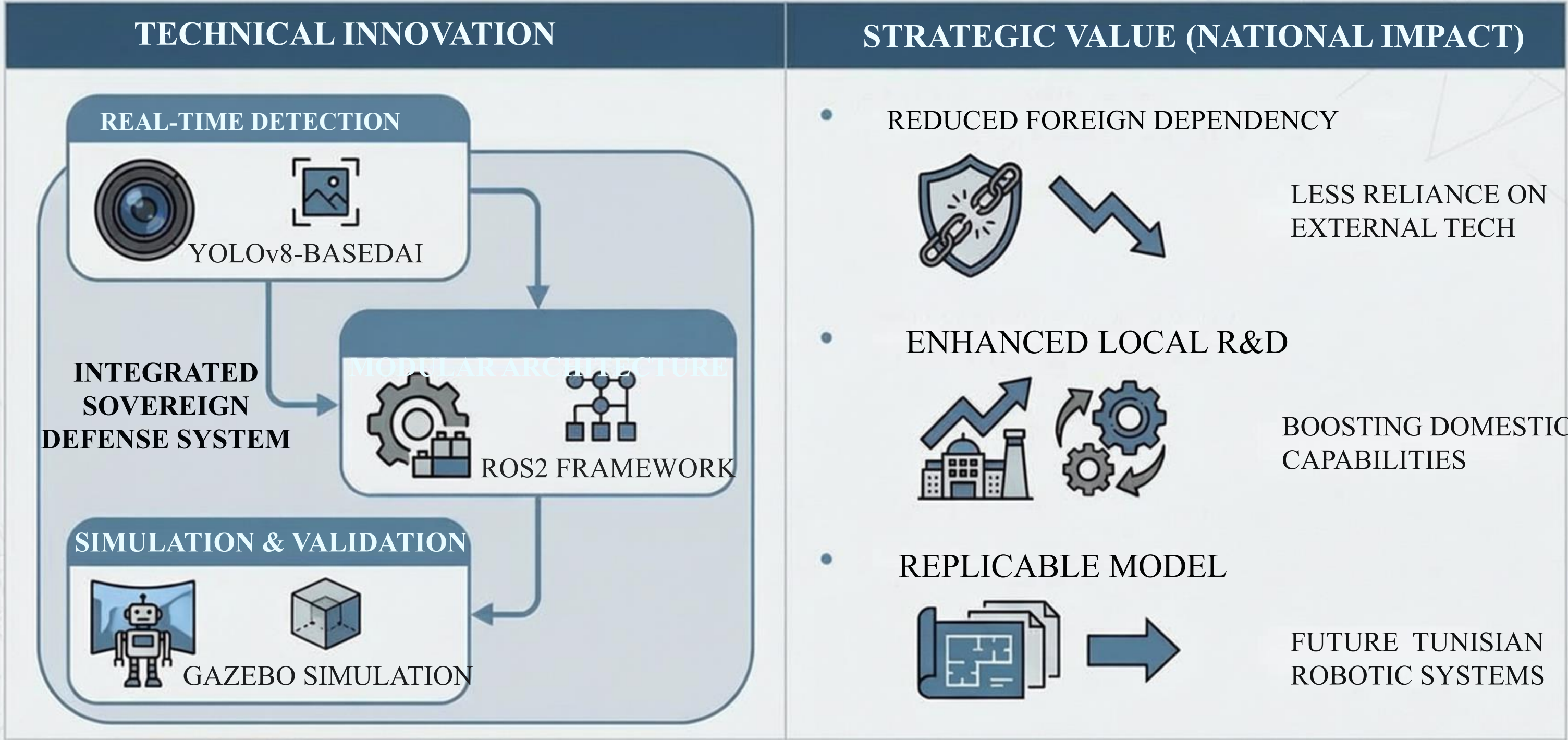
- Multi-Modal Data Integration
- Real-Time Environment Perception



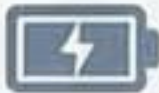
ROS2-NATIVE ARCHITECTURE

- Scalable & Modular Software
- Distributed Communication
- Real-Time Control Systems

TECHNICAL AND STRATEGIC VALUE



CDCF SPECIFICATION TABULATION

FUNCTIONAL PERFORMANCE (FP) SPECIFICATIONS		FUNCTIONAL COMPONENT(FC) SPECIFICATIONS	
	FP1: Detect and track dynamic targets		FC1: Handheld Controller
			FC2: Battery System
	FP2: Integration of navigation system		FC3: ISO Fire- Safety Compliance
			FC4: energy management
	FP3: Rugged Chassis		FC5:The robot have a reasonable volume and weight.
			FC6: Target BOM Cost (< 5 k USD)

Note:CDCF=Critical Design & Component Framework .

SYSTEM REQUIREMENTS & PERFORMANCE BENCHMARKS



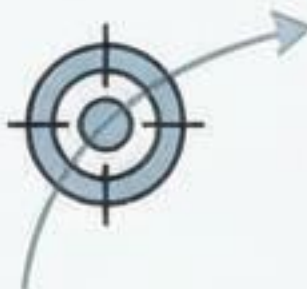
OBJECT DETECTION



Different military
object detection



TRACKING ACCURACY



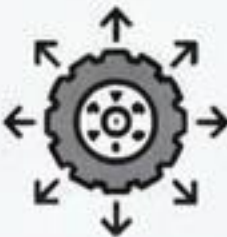
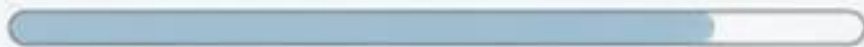
3 cm
Accuracy



ENDURANCE & MOBILITY



- 8-hour endurance



- Omnidirectional mobility

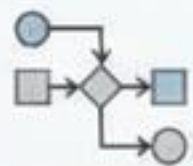


AUTONOMY & VALIDATION



- Gazebo – validated autonomy

-



Hardware
Selection

Software
Tuning

Test Scenarios

C

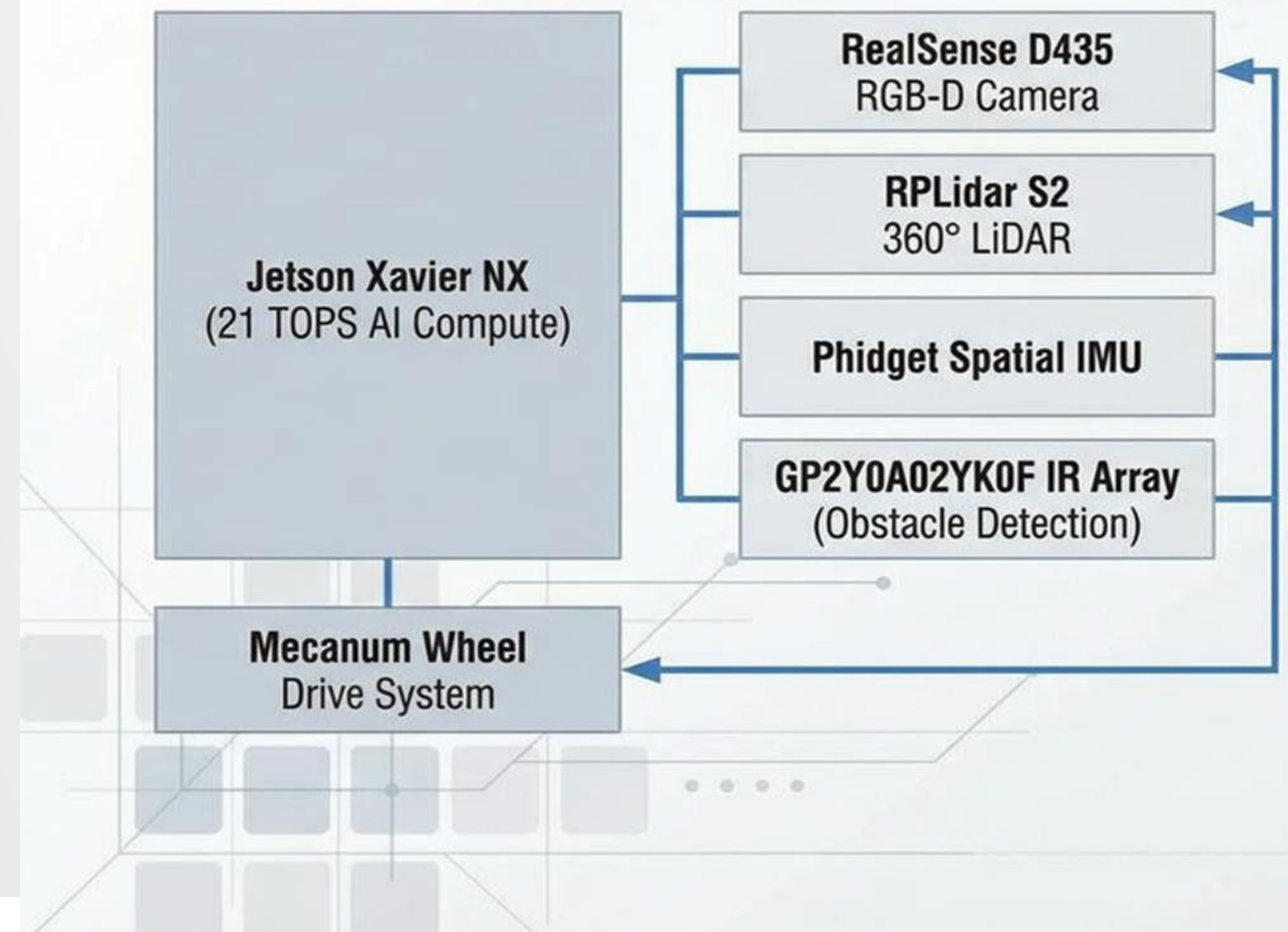
D

H/S architecture

E

F

NOMAD SAMURAI: System Architecture



Camera Configuration



Intel RealSense D435

RGB-D Camera providing 3D perception for the SAMURAI system



Simultaneous RGB & Depth

Captures both color and depth data concurrently, enabling rich 3D scene understanding



Obstacle Detection

Enables real-time identification of obstacles within the rover's operational range



Depth-aware Threat Localization

Provides crucial depth information for accurate threat localization and response



LiDAR and Navigation Sensors

RPLiDAR S2



- 360° Laser Scanner
- 2D Spatial Mapping
- Obstacle Detection

Generates occupancy grids and environmental maps vital for navigation and obstacle avoidance in the SAMURAI system.

Phidget IMU



- Motion Tracking
- Orientation Detection
- GPS-Denied Operation

Measures acceleration, angular velocity, and orientation, providing essential data for localization in environments where GPS signals are unavailable.

Infrared Proximity Detection

Detection Range: 20 cm - 150 cm



IR Sensor Range Comparison	
Sensor family	Range of distance
GP2Y0A02YK0F*	20cm – 150cm
GP2Y0A41SK0F*	4cm – 30cm
GP2Y0A21*	10cm – 80cm
GP2Y0A710K0F*	100cm – 550cm



 Sensor Model

GP2Y0A02YK0F

 Function

Object distance estimation

 Integration

NOMAD rover sensor suite



Computation Platform Jetson Xavier NX



AI Performance

21 TOPS (Trillion Operations Per Second)

Maximum AI performance



CPU Architecture

6-core NVIDIA Carmel ARM CPU
for efficient processing



Graphics Processor

384-core NVIDIA Volta GPU
with Tensor Cores



Primary Applications

- Real-time object detection
- Complex robotic computations
- Edge AI inference

ROS 2 Humble Framework

The SAMURAI system leverages **ROS 2 Humble** as its foundational software architecture, providing a robust and modular framework for robotic development.



Distributed Architecture

Nodes perform specific tasks, enhancing modularity and system reliability



Real-time Communication

DDS protocol enables asynchronous, scalable data exchange



Enhanced Reusability

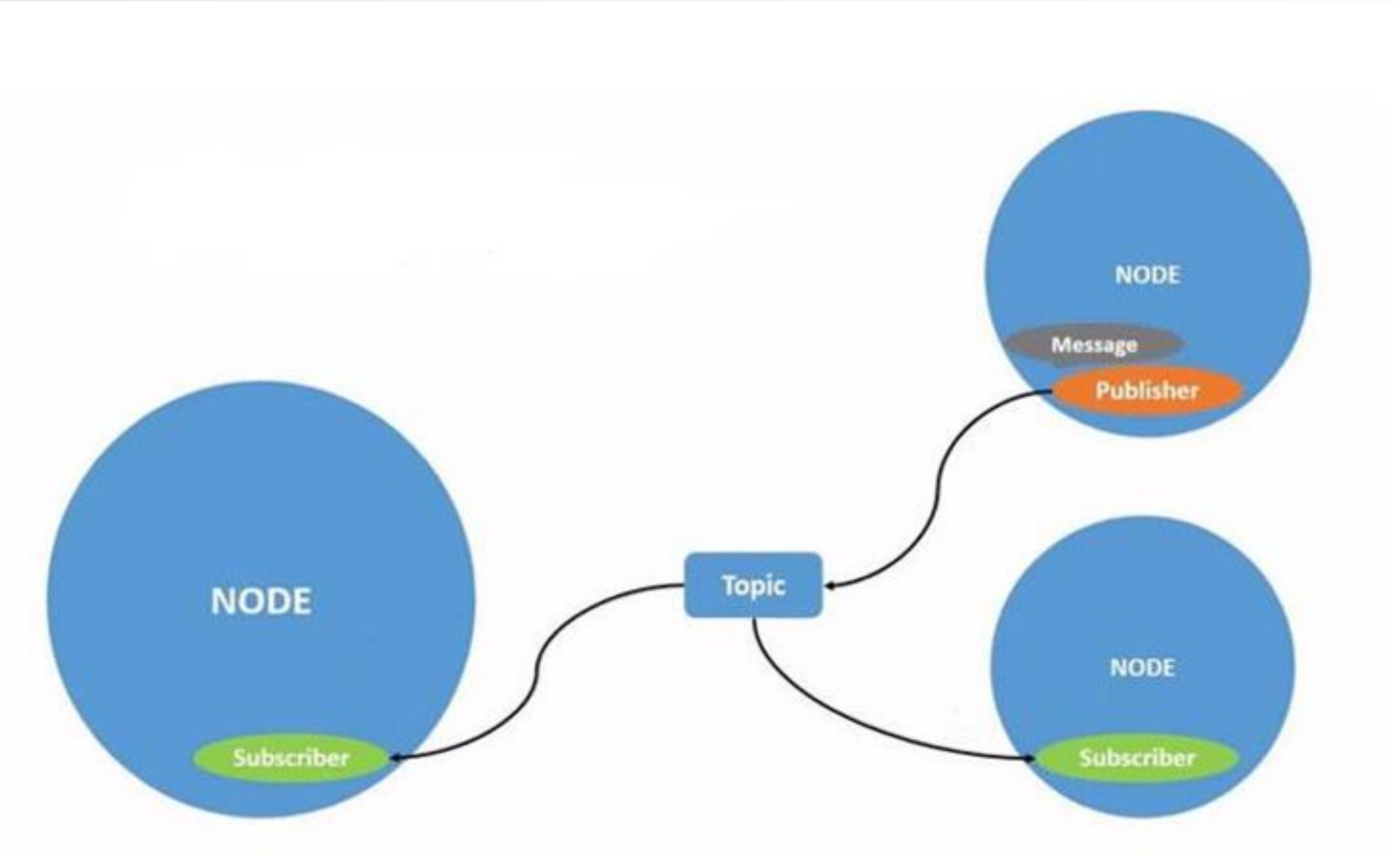
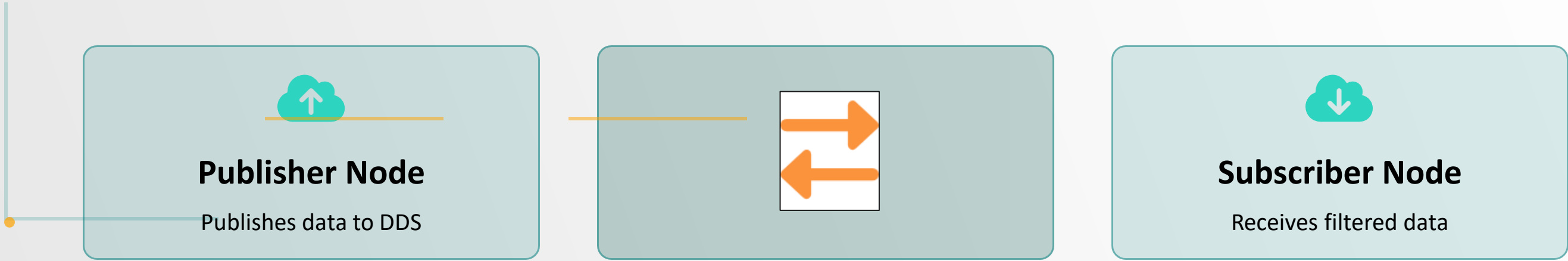
Modular components can be easily repurposed for different applications



Flexible Execution

Adaptable to various computing resources and deployment scenarios

Communication Architecture



Simulation and Visualization Tools



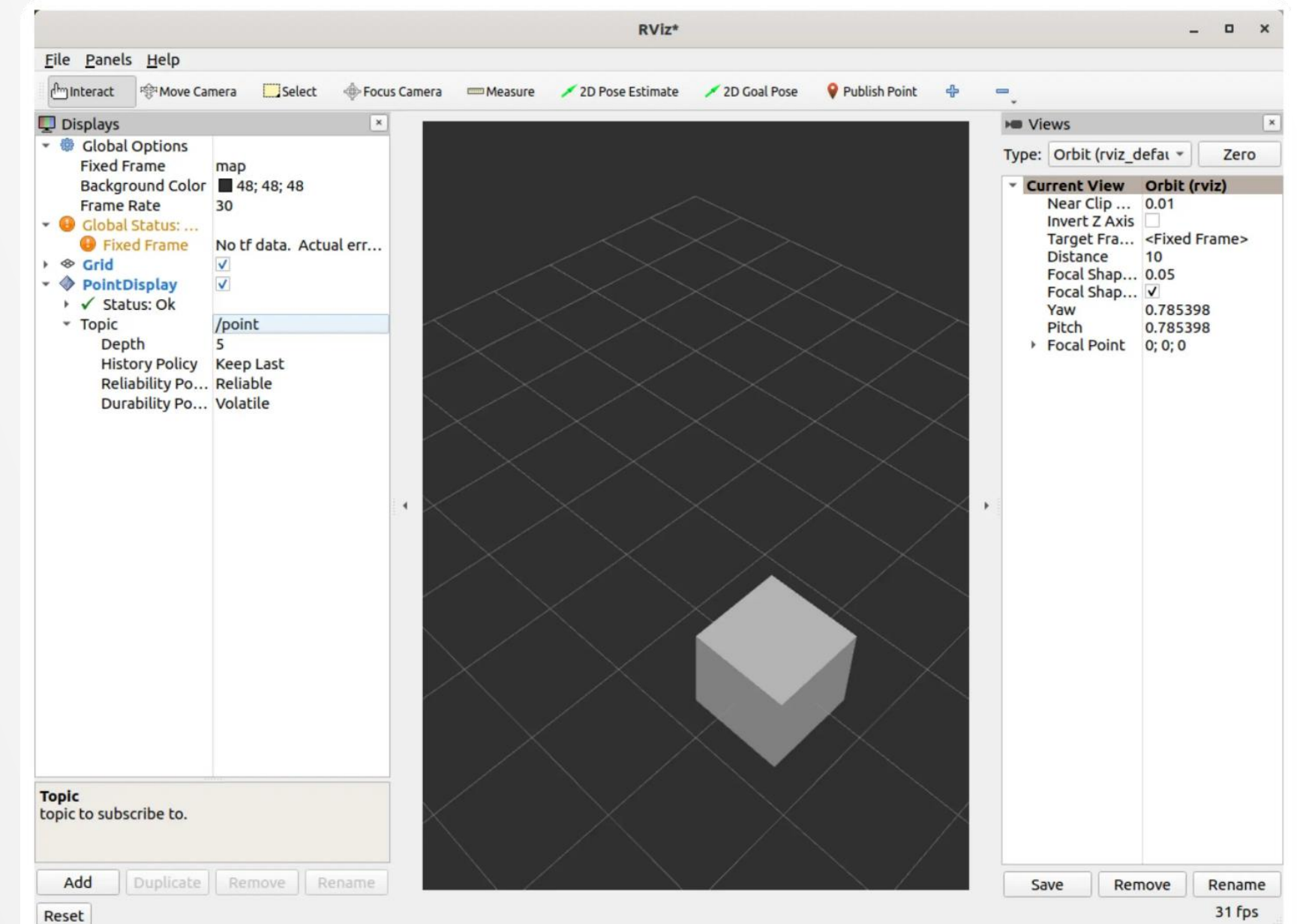
Gazebo

- ✓ Realistic physics-based simulation environment
- ✓ Used for virtual testing of robot behaviors
- ✓ Integration with ROS 2 for realistic testing



RViz2

- ✓ 3D visualization tool for robotics
- ✓ Debugging of robot states and sensor data
- ✓ Visualization of point clouds, images, and robot models



RViz2 Interface showing 3D visualization of robot and environment

Robot Description URDF

What is URDF?

URDF (Unified Robot Description Format) defines robot's mechanical structure and geometry, enabling visualization, trajectory planning, and simulation in ROS 2.

Key Capabilities

- ✓ Enables accurate 3D visualization in RViz2
- ✓ Supports trajectory planning and navigation
- ✓ Facilitates simulation in Gazebo

URDF Structure Example

```
<robot>  name="nomad_rover"
  <link>  name="base_link"
  <link>  name="wheel_1"
  <link>  name="wheel_2"
  <joint> name="base_to_wheel_1"
```

Integration with SAMURAI

URDF models are central to SAMURAI, providing a unified description of the robot's structure for accurate simulation, visualization, and control across ROS 2 packages.

YOLOv8 Object Detection Pipeline



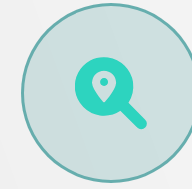
Camera Input

RGB-D camera feed



YOLOv8 Inference

Neural network processing



Object Detection

Class labels & bounding boxes



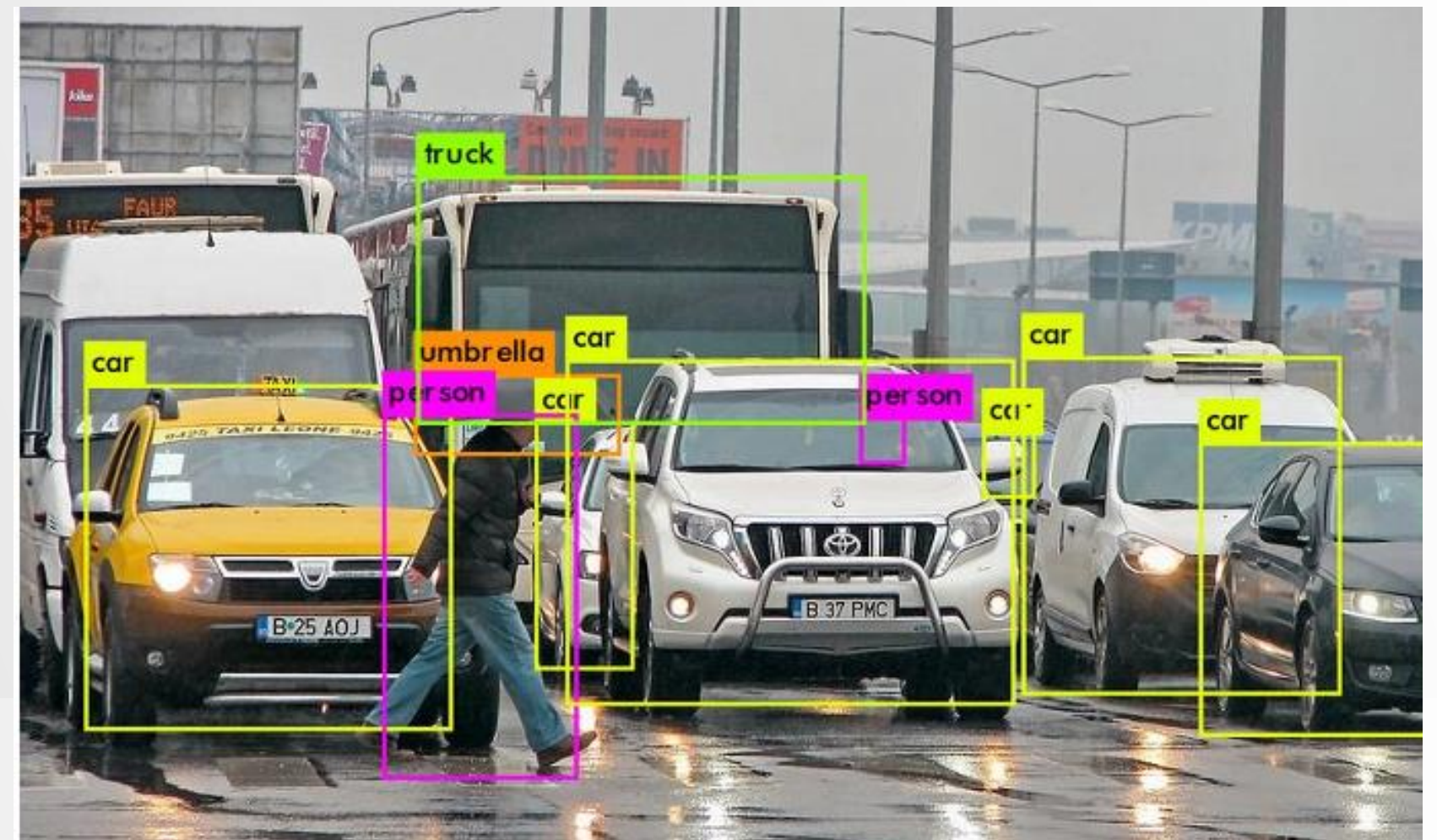
Output

Annotated images



Key Features

- ✓ Real-time object detection with high accuracy
- ✓ Specialized model for military-relevant classes
- ✓ Integration with ROS 2 framework



D

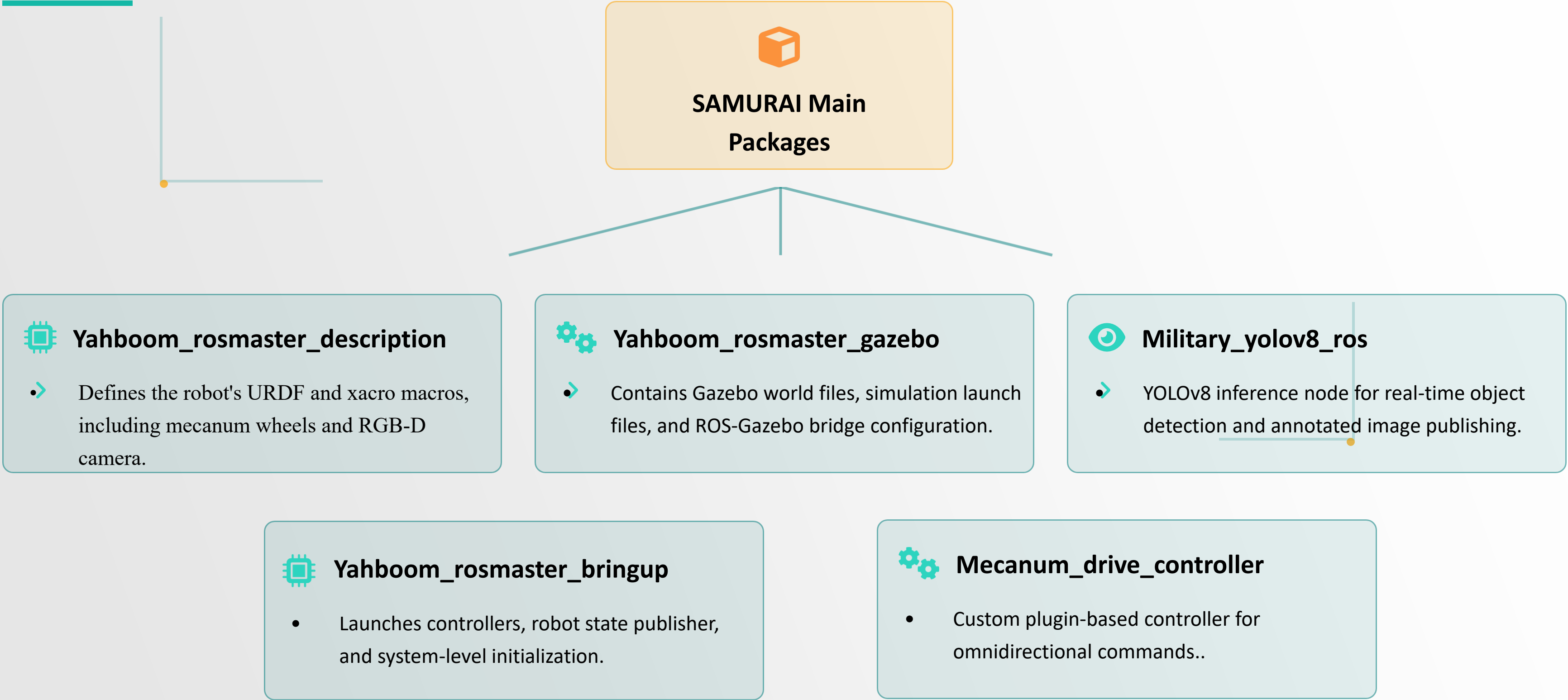
E

F

Realization and Simulation



ROS 2 Package Architecture



Mecanum Drive Controller



Plugin Architecture

Operates as a plugin for ros2_control, managed by controller_manager



Message Type

Uses geometry_msgs/msg/TwistStamped for time-synchronized control



Command Interface

Receives velocity commands via /mecanum_drive_controller/cmd_vel topic



Configuration

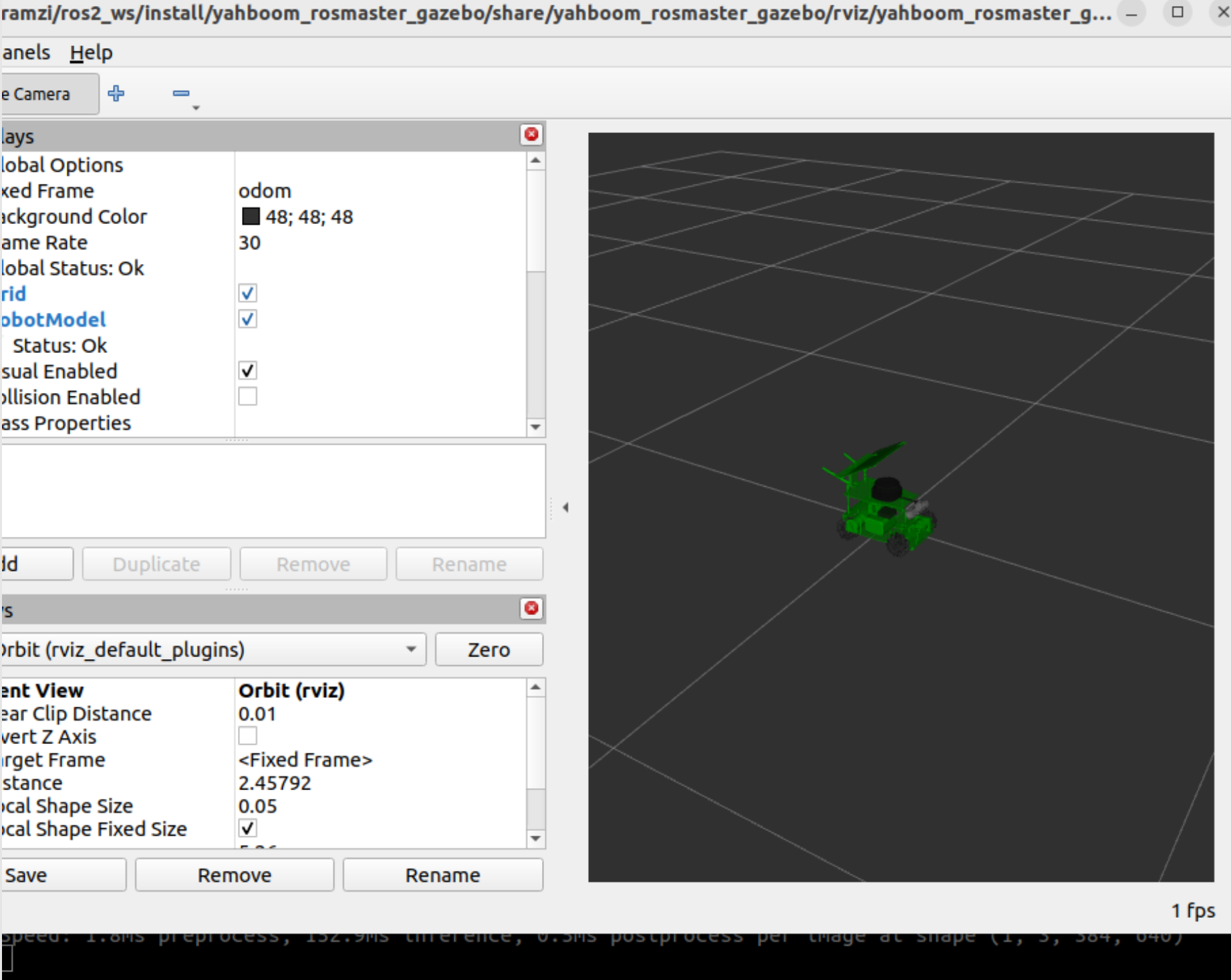
Parameters (wheel joints, radius, separation, etc.) configured via YAML file

Mecanum Drive Controller

```
ramzi@ramzi-virtual-machine: ~/ros2_ws 101x48
ramzi@ramzi-virtual-machine:~/ros2_ws$ ros2 topic pub /mecanum_drive_controller/cmd_vel geometry_msgs
/msg/TwistStamped "{header: {stamp: {sec: $(date +%s), nanosec: 0}, frame_id: ''}, twist: {linear: {x
: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}}"
publisher: beginning loop
publishing #1: geometry_msgs.msg.TwistStamped(header=std_msgs.msg.Header(stamp=builtin_interfaces.msg
.Time(sec=1757243133, nanosec=0), frame_id=''), twist=geometry_msgs.msg.Twist(linear=geometry_msgs.ms
g.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0)))

publishing #2: geometry_msgs.msg.TwistStamped(header=std_msgs.msg.Header(stamp=builtin_interfaces.msg
.Time(sec=1757243133, nanosec=0), frame_id=''), twist=geometry_msgs.msg.Twist(linear=geometry_msgs.ms
g.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0)))

publishing #3: geometry_msgs.msg.TwistStamped(header=std_msgs.msg.Header(stamp=builtin_interfaces.msg
.Time(sec=1757243133, nanosec=0), frame_id=''), twist=geometry_msgs.msg.Twist(linear=geometry_msgs.ms
g.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0)))
```



```

publishing #33: geometry_msgs.msg.TwistStamped(header=std_msgs.msg.Header(stamp=builtin_interfaces.ms
g.Time(sec=1757265435, nanosec=0), frame_id=''), twist=geometry_msgs.msg.Twist(linear=geometry_msgs.m
sg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=-0.4)))

publishing #34: geometry_msgs.msg.TwistStamped(header=std_msgs.msg.Header(stamp=builtin_interfaces.ms
g.Time(sec=1757265435, nanosec=0), frame_id=''), twist=geometry_msgs.msg.Twist(linear=geometry_msgs.m
sg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=-0.4)))

publishing #35: geometry_msgs.msg.TwistStamped(header=std_msgs.msg.Header(stamp=builtin_interfaces.ms
g.Time(sec=1757265435, nanosec=0), frame_id=''), twist=geometry_msgs.msg.Twist(linear=geometry_msgs.m
sg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=-0.4)))

publishing #36: geometry_msgs.msg.TwistStamped(header=std_msgs.msg.Header(stamp=builtin_interfaces.ms
g.Time(sec=1757265435, nanosec=0), frame_id=''), twist=geometry_msgs.msg.Twist(linear=geometry_msgs.m
sg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=-0.4)))

publishing #37: geometry_msgs.msg.TwistStamped(header=std_msgs.msg.Header(stamp=builtin_interfaces.ms
g.Time(sec=1757265435, nanosec=0), frame_id=''), twist=geometry_msgs.msg.Twist(linear=geometry_msgs.m
sg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=-0.4)))

^Cramzi@ramzi-virtual-machine:~/ros2_ws$ ros2 topic pub /mecanum_drive_controller/cmd_vel geometry_ms
gs/msg/TwistStamped "{
  header: {
    stamp: {sec: `date +%s`, nanosec: 0},
    frame_id: ''
  },
  twist: {
    linear: {
      x: 0.0,
      y: 0.0,
      z: 0.0
    },
    angular: {
      x: 0.0,
      y: 0.0,
      z: -0.4
    }
  }
}"

```

Mobility Validation Results

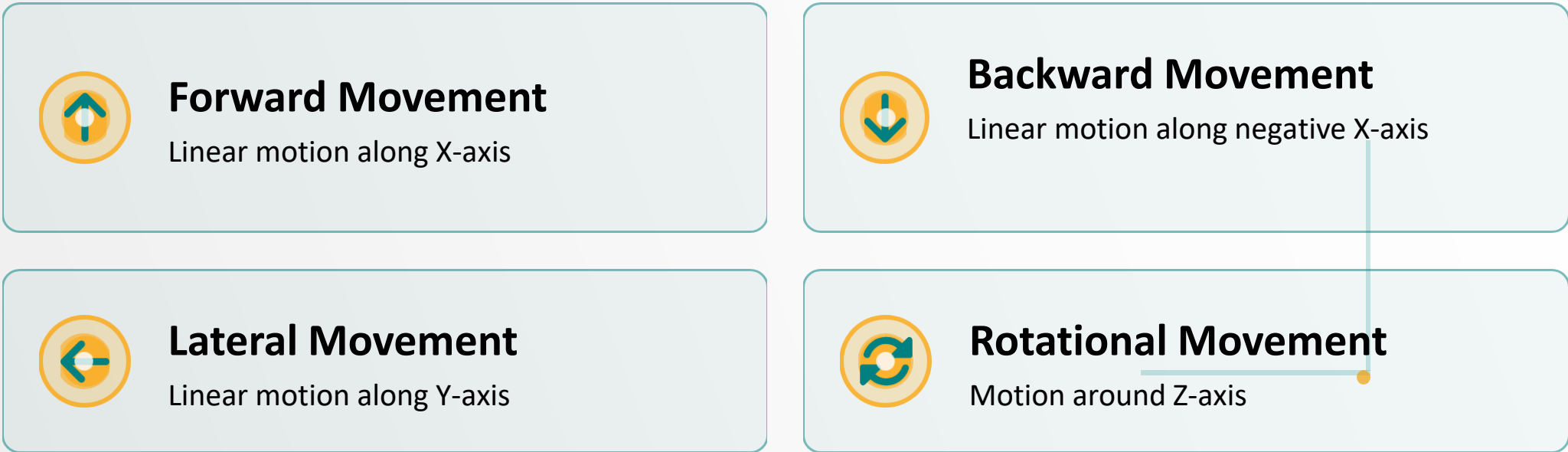
Key Validation Results

- ✓ Omnidirectional movement successfully demonstrated in simulation
- ✓ Precise control of forward, backward, lateral, and rotational maneuvers
- ✓ Accurate translation of velocity commands into wheel signals
- ✓ Visual confirmation in Gazebo

Technical Achievement

The **mecanum_drive_controller** successfully closed the control loop, translating high-level velocity commands into precise wheel movements.

Omnidirectional Movement Patterns

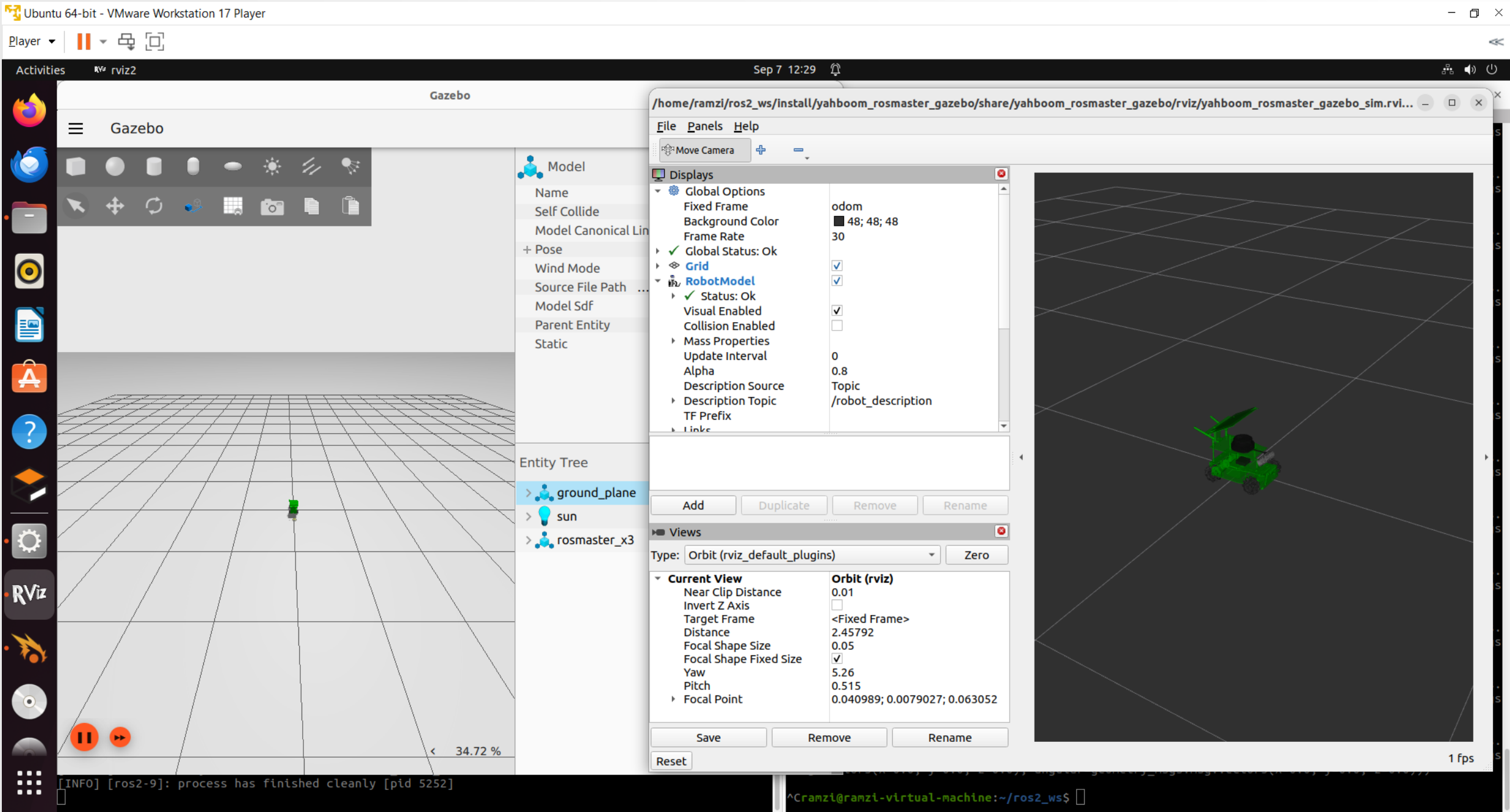


Simulation Environment Setup



Gazebo Simulation

Realistic physics-based simulation environment for testing without physical hardware

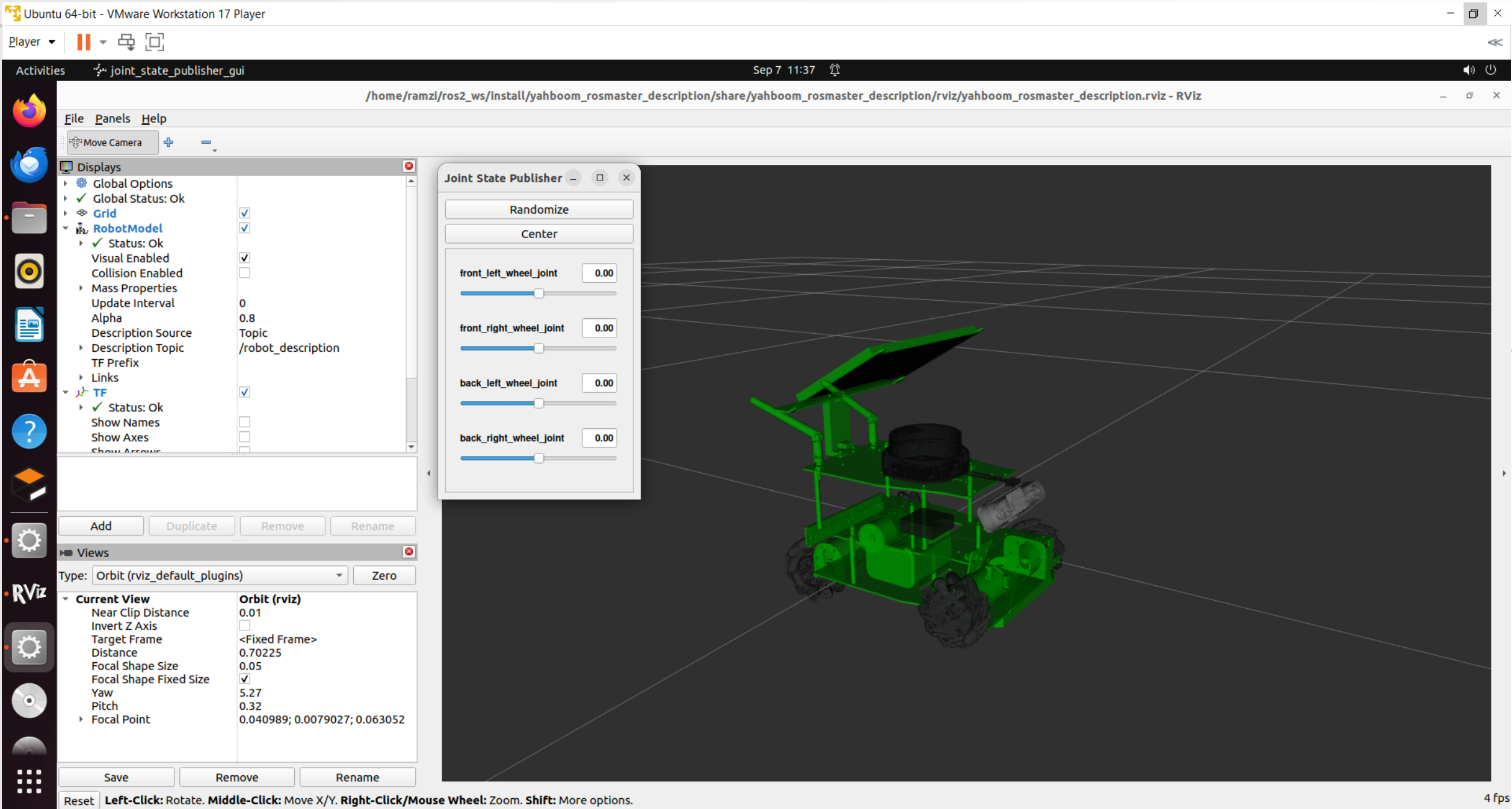


Simulation Environment Setup



Rviz Simulation

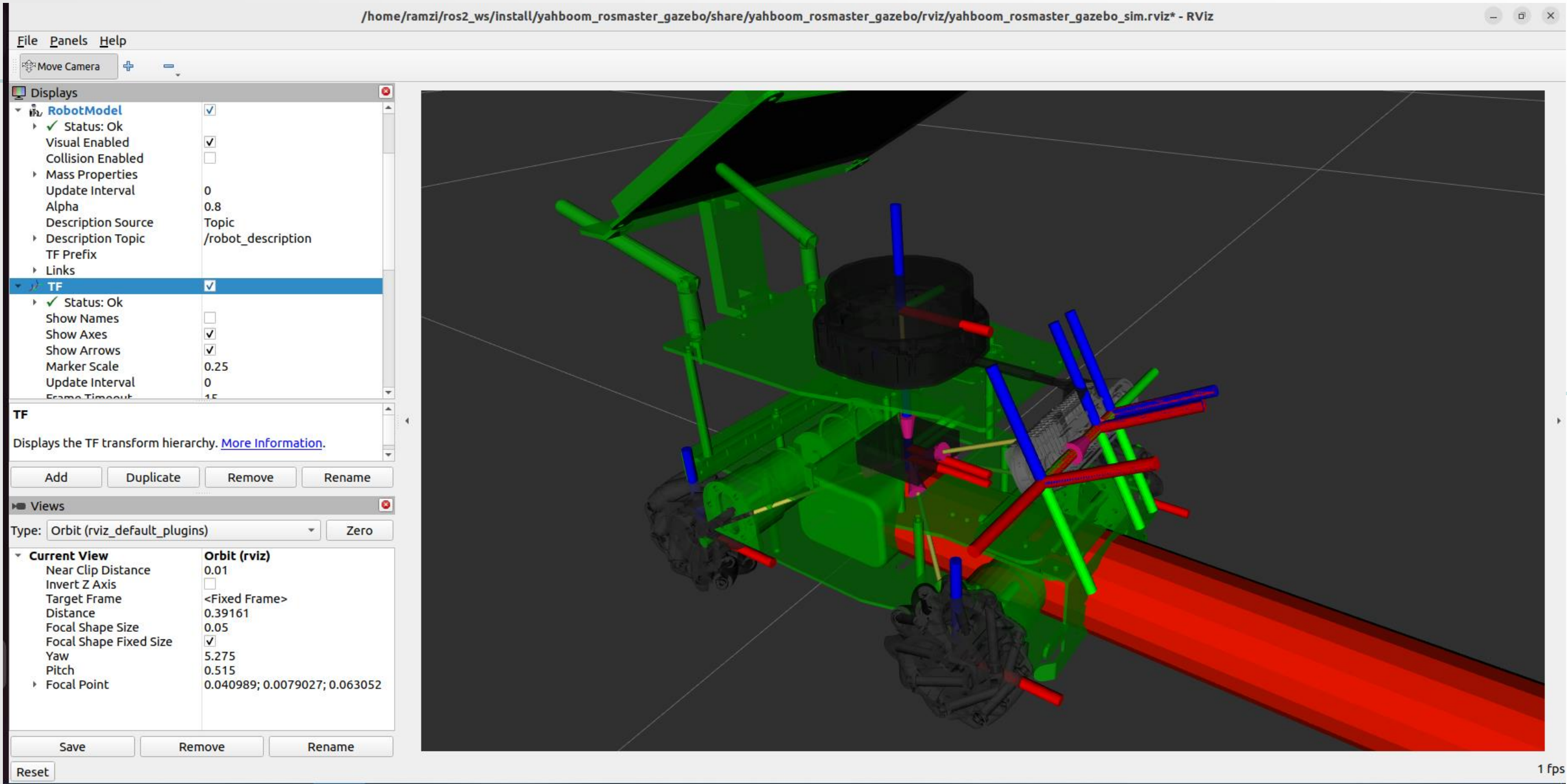
By launching the URDF file using XACRO file the result is :



Simulation Environment Setup



Coordinate frames of the Robot .

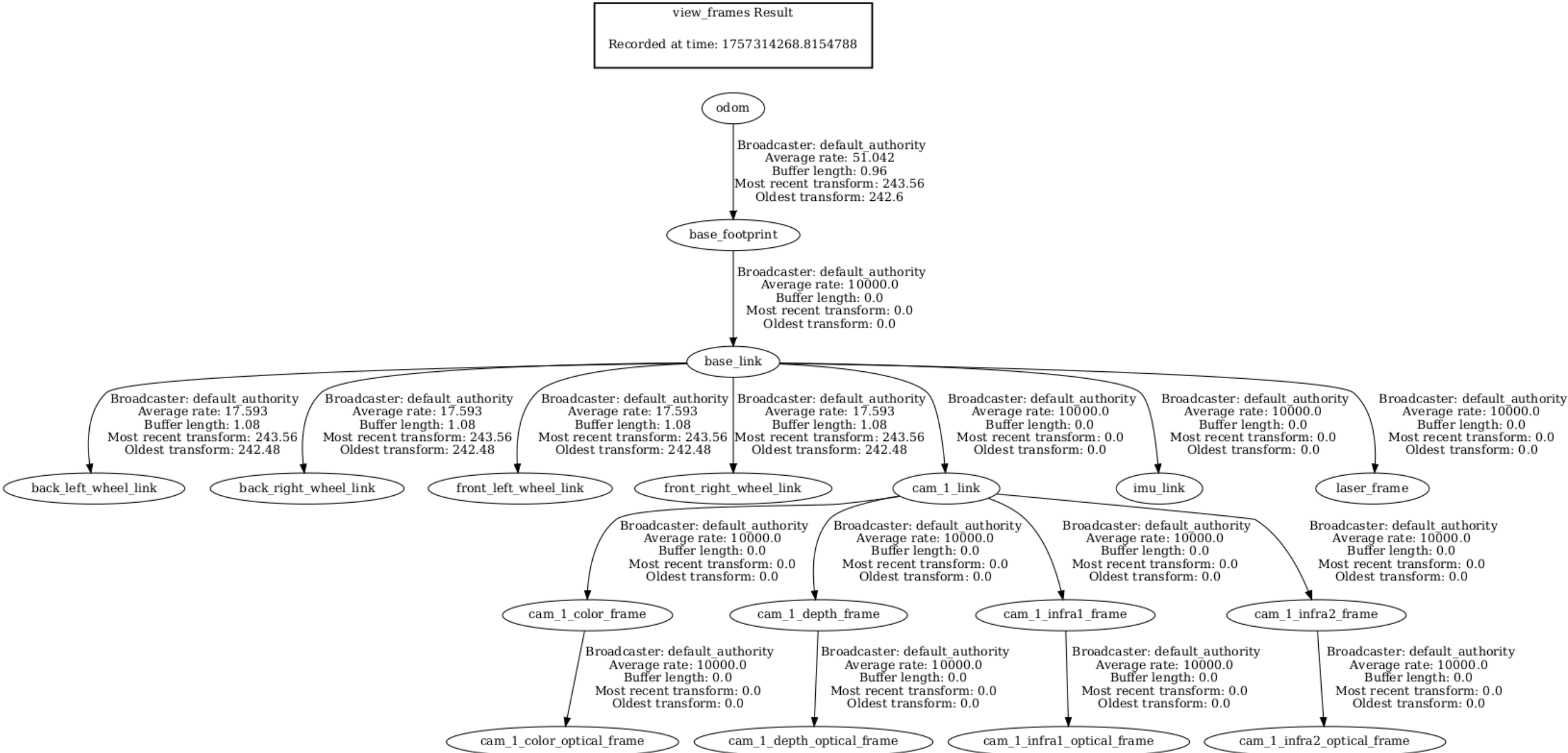


Simulation Environment Setup



TF check

the TF checkbox to turn on the axes



Simulation Environment Setup



Ros2 controllers List

```
ramzi@ramzi-virtual-machine: ~/ros2_ws 101x23
ramzi@ramzi-virtual-machine:~/ros2_ws$ ros2 control list_controllers
joint_state_broadcaster  joint_state_broadcaster/JointStateBroadcaster  active
mecanum_drive_controller mecanum_drive_controller/MecanumDriveController  active
ramzi@ramzi-virtual-machine:~/ros2_ws$
```

- **Joint_state_broadcaster:** Publishes joint states to /joint_states, allowing RViz and other tools to visualize robot motion.
- **Mecanum_drive_controller:** the controller plugin receives velocity commands and translates them into wheel joint efforts for omnidirectional movement.

Sensor Integration Validation ROS-Gazebo bridge

The SAMURAI system integrated data from its multi-sensor suite in ROS 2 and RViz2:



LiDAR Scan Visualization

Real-time 2D mapping data from RPLiDAR S2 visualized in RViz2 for navigation and obstacle detection



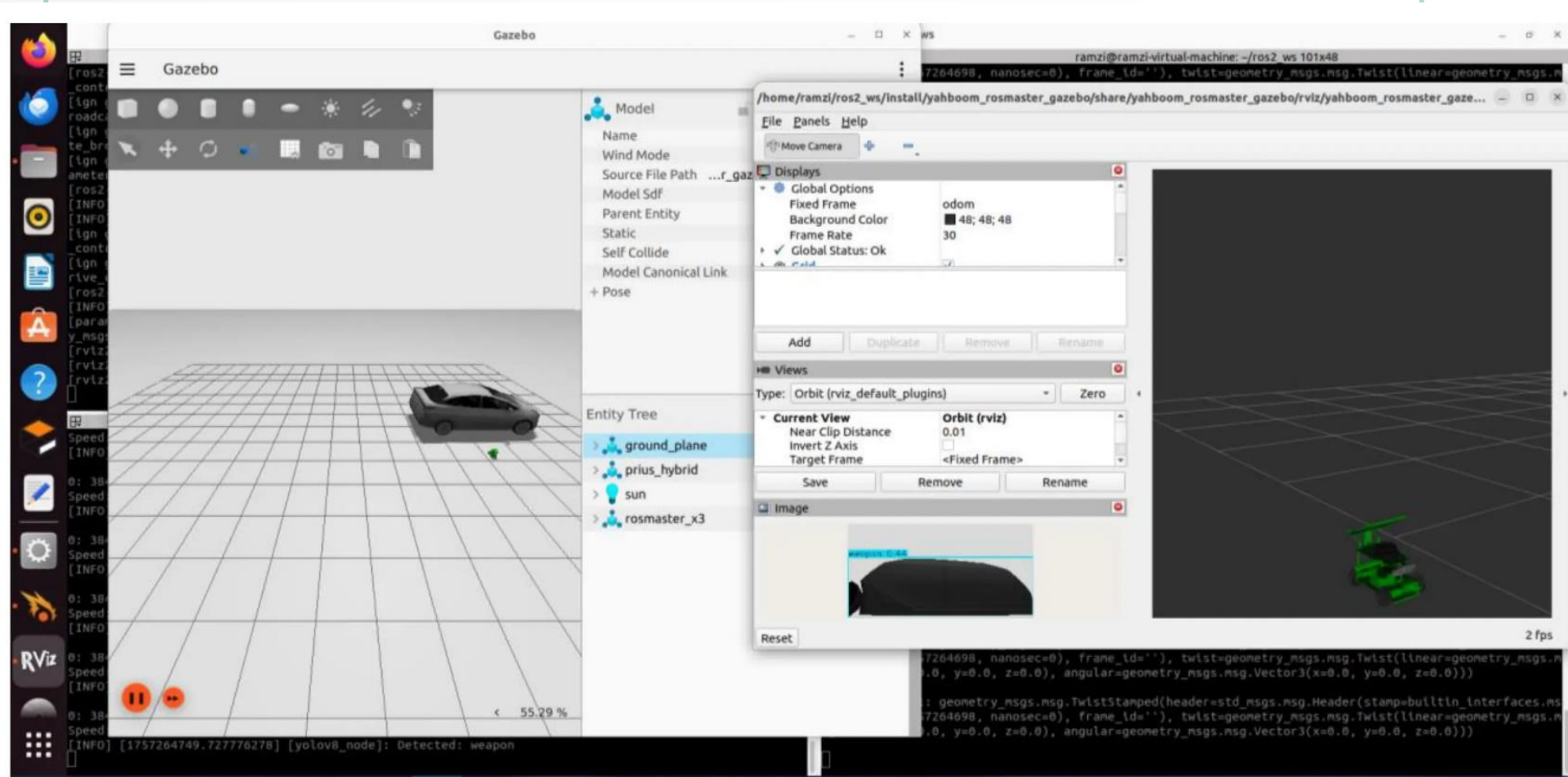
IMU Data Stream

Phidget MOT0110 IMU provided angular velocity and linear acceleration data for localization



RGB-D Camera Feed

Intel RealSense D435 data bridged to ROS 2, enabling perception modules like modules like YOLOv8



RViz2 visualization showing sensor data integration

Sensor Integration Validation ROS-Gazebo bridge

Sensor topics

ros2 topic echo /mecanum_drive_controller/cmd_vel

ros2 topic echo /joint_states

ros2 topic echo /mecanum_drive_controller/odom

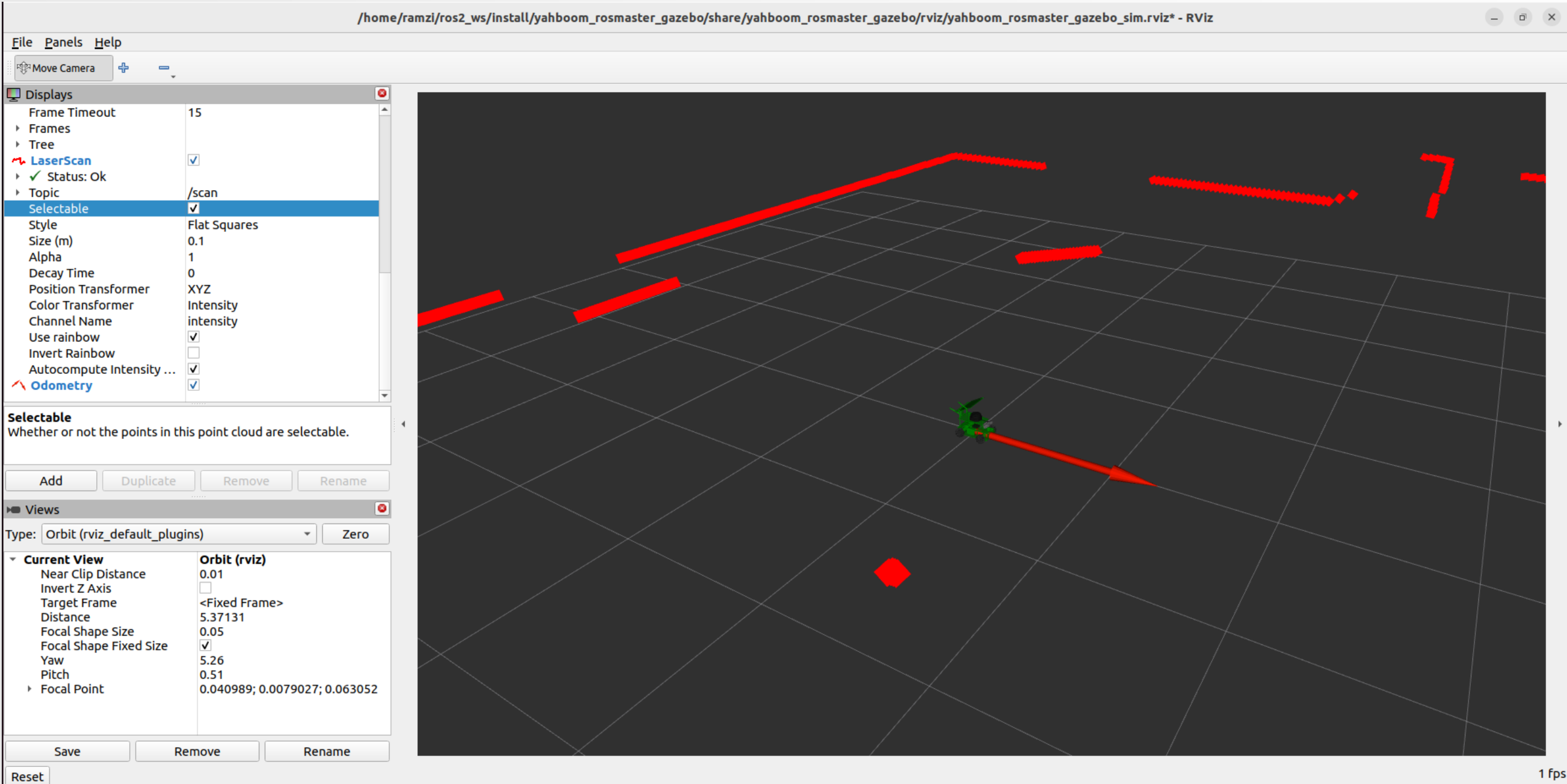
ros2 topic echo /imu/data

ros2 topic echo /scan displays

Sensor Integration Validation ROS-Gazebo bridge



LIDAR Scan



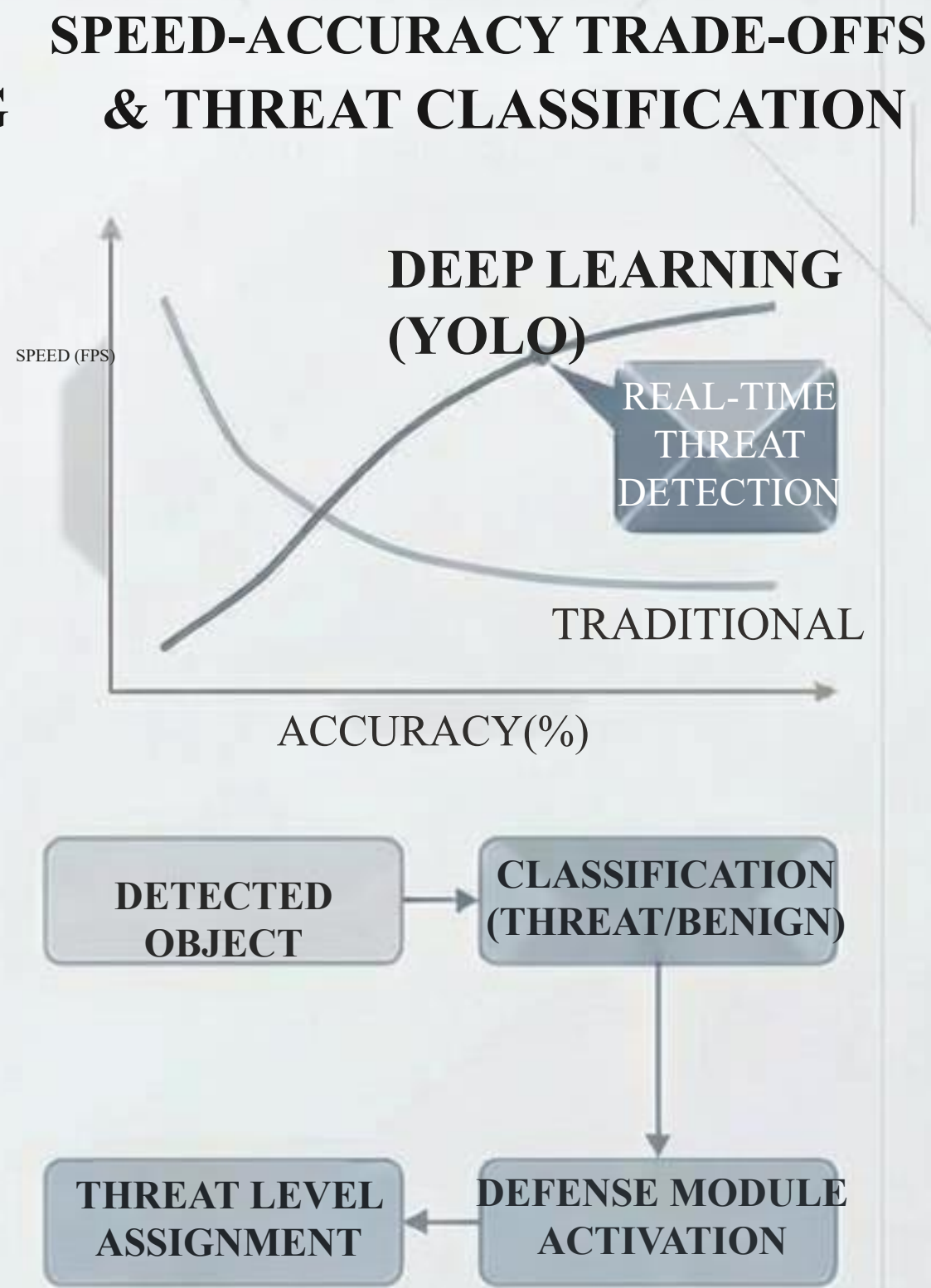
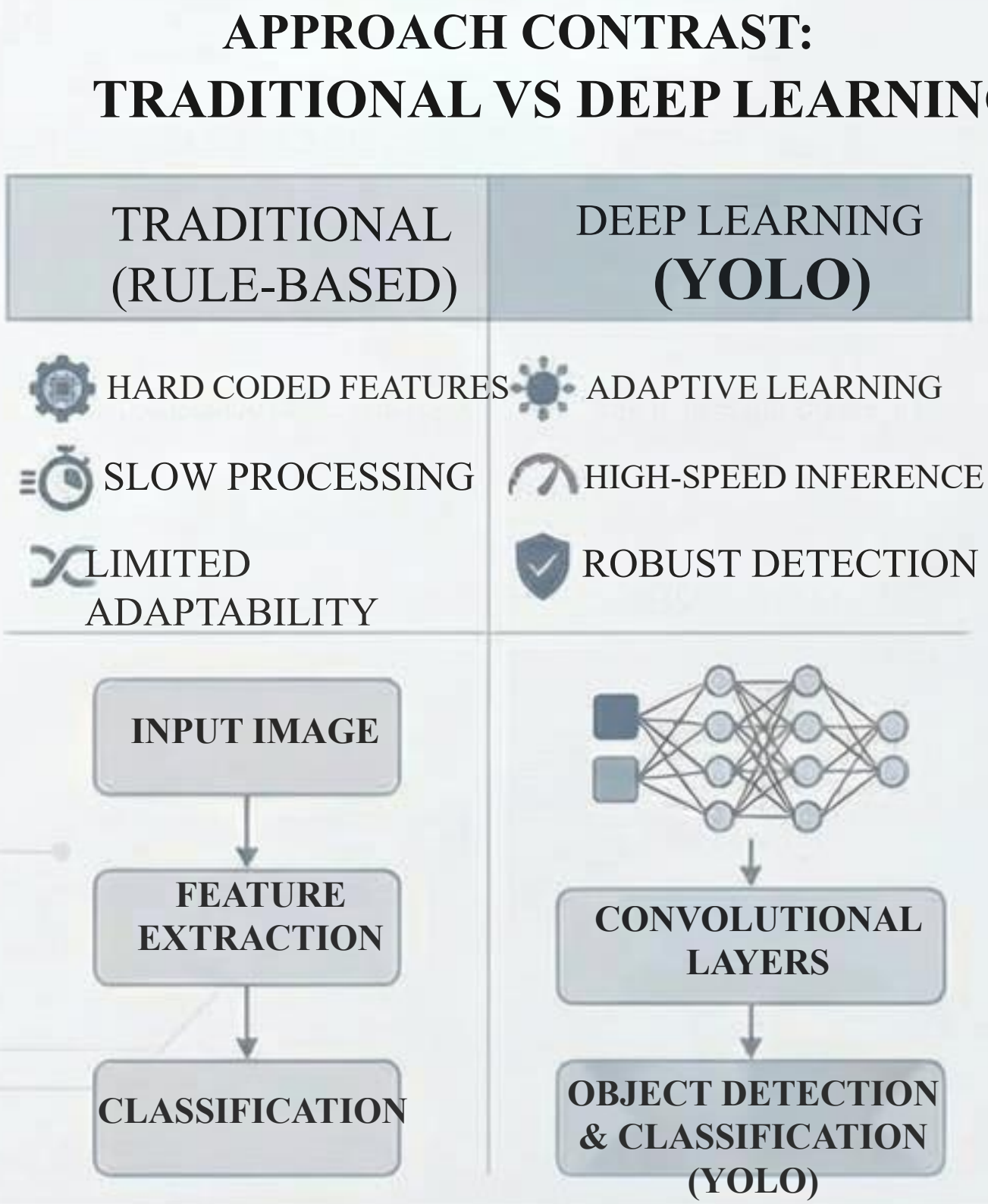
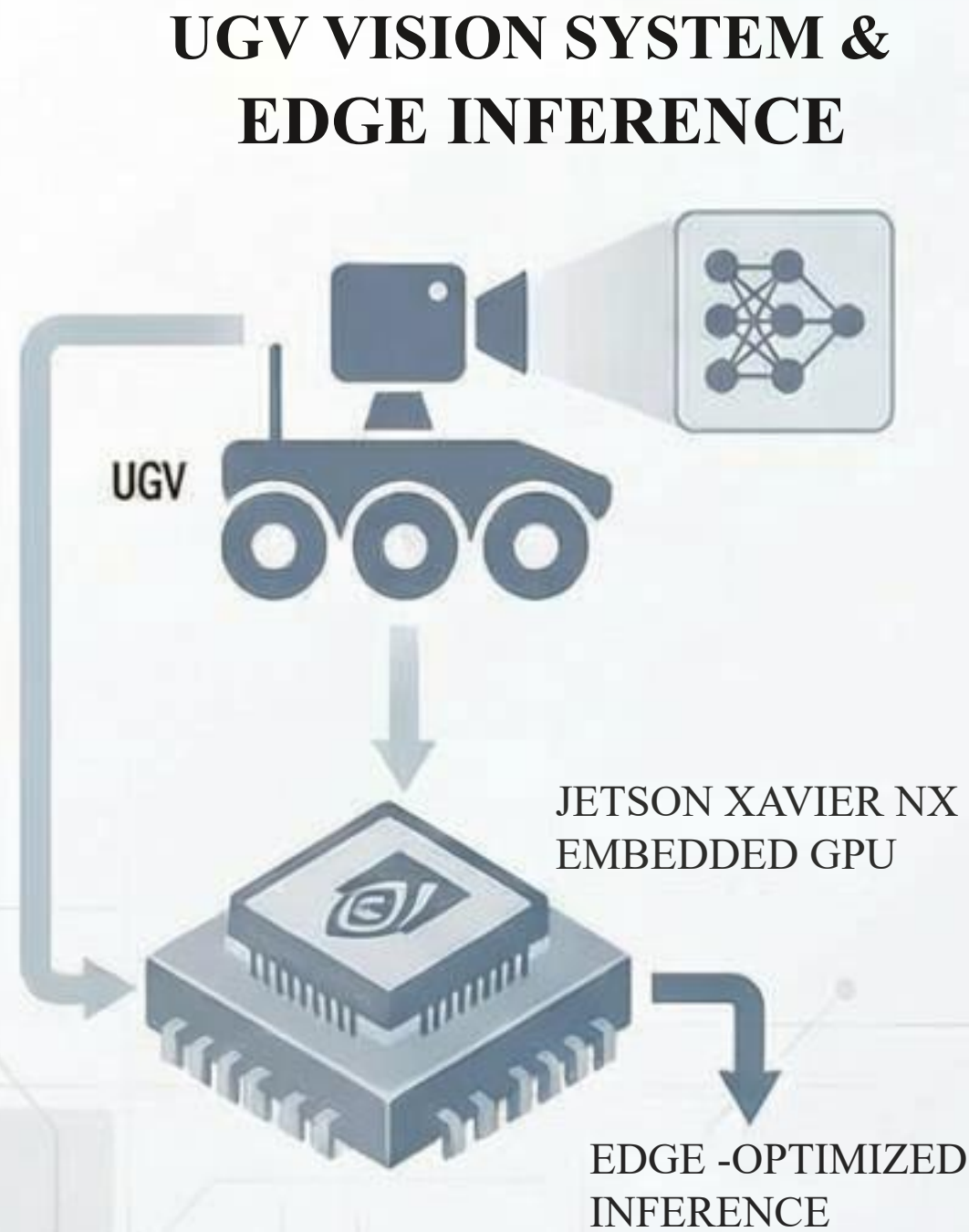
Model Selection Justification: YOLOv8 for Embedded Deployment

Object detection can be done using various approaches such as:

1. SSD (Single Shot Detector)
2. R-CNN (Regional Convolutional Neural Network)
3. Fast R-CNN
4. Faster R-CNN
5. Mask R-CNN

Although these approaches have solved data limitation and modeling problems in object detection, they are not able to detect objects in a single algorithm cycle. The YOLO algorithm has gained popularity due to its superior performance compared to the aforementioned object detection techniques.

COMPUTER VISION INTEGRATION & REAL-TIME OBJECT DETECTION



YOLOv8 Model Training Process

Training Pipeline



Custom Dataset Preparation

26,315 labeled images across 12 military-relevant classes



Model Configuration

YOLOv8 architecture optimized for edge AI deployment



Training Execution

Supervised learning with bounding box annotation



Validation & Optimization

Performance analysis and hyperparameter tuning

Training Interface

```
YOLOv8n summary: 129 layers, 3,013,188 parameters, 3,013,172 gradients, 8.2 GFLOPs
Freezing layer 'model.22.dfl.conv.weight'
train: Fast image access (ping: 0.0±0.0 ms, read: 134.3±88.0 MB/s, size: 123.9 KB)
train: Scanning /home/ranzi/ros2_ws/src/yahboom_rosmaster/military_yolov8_ros/military_yolov8_ros/data/train/labels.cache... 21978 images, 294 backgrounds, 0 corrupt: 100%
train: Scanning /home/ranzi/ros2_ws/src/yahboom_rosmaster/military_yolov8_ros/military_yolov8_ros/data/train/labels.cache... 21978 images, 294 backgrounds, 0 corrupt: 100%
t/s 0.0s
WARNING: box and segment counts should be equal, but got len(segments) = 22, len(boxes) = 43336. To resolve this only boxes will be used and all segments will be removed. To avoid this please supply either a detect or segment dataset, not a detect-segment mixed dataset.
val: Fast image access (ping: 0.0±0.0 ms, read: 128.7±86.0 MB/s, size: 113.2 KB)
val: Scanning /home/ranzi/ros2_ws/src/yahboom_rosmaster/military_yolov8_ros/military_yolov8_ros/data/val/labels.cache... 2941 images, 273 backgrounds, 0 corrupt: 100%
val: Scanning /home/ranzi/ros2_ws/src/yahboom_rosmaster/military_yolov8_ros/military_yolov8_ros/data/val/labels.cache... 2941 images, 273 backgrounds, 0 corrupt: 100%
Plotting labels to /home/ranzi/ros2_ws/runs/detect/train8/labels.jpg...
optimizer: 'optimizer=auto' found, ignoring 'lr=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr' and 'momentum' automatically...
optimizer: AdamW(lr=0.000025, momentum=0.9) with parameter groups 57 weight(decay=0.0), 64 weight(decay=0.0005), 63 bias(decay=0.0)
Image sizes 640 train, 640 val
Using 0 dataloader workers
Logging results to /home/ranzi/ros2_ws/runs/detect/train8
Starting training for 5 epochs...

Epoch   GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
1/5      8G       3.469    4.931    4.151      41         640: 31% 431/1374 0.11t/s 1:19:49~3:01:01

ranzi@ranzi-virtual-machine: ~/ros2_ws$ ros2 launch yahboom_rosmaster_gazebo yahboom_rosmaster_gazebo.launch.py
Starting >>> yahboom_rosmaster_system_tests
Finished <<< yahboom_rosmaster_gazebo [0.47s]
Starting >>> mecanum_drive_controller
Finished <<< yahboom_rosmaster_system_tests [24.0s]
Starting >>> military_yolov8_ros
[25.980s] WARNING:colcon.colcon_ros.task.anent.python.build:Package 'military_yolov8_ros' doesn't explicitly install a marker in the package index (colcon-ros currently does it implicitly but that fallback will be removed in the future)
Finished <<< military_yolov8_ros [3.54s]
Starting >>> yahboom_rosmaster
Finished <<< yahboom_rosmaster [2.73s]
Finished <<< mecanum_drive_controller [45.4s]

Summary: 7 packages finished [46.7s]
ranzi@ranzi-virtual-machine:~/ros2_ws$ --allow-overriding mecanum_drive_controller
--allow-overriding: command not found
ranzi@ranzi-virtual-machine:~/ros2_ws$ --allow-overriding mecanum_drive_controller
--allow-overriding: command not found
ranzi@ranzi-virtual-machine:~/ros2_ws$

[ign gazebo-3] [Msg] Publishing scene information on [/world/default/scene/info]
[ign gazebo-3] [Msg] Publishing entity deletions on [/world/default/scene/deletion]
[ign gazebo-3] [Msg] Publishing state changes on [/world/default/state]
[ign gazebo-3] [Msg] Publishing pose messages on [/world/default/pose/info]
[ign gazebo-3] [Msg] Publishing dynamic pose messages on [/world/default/dynamic_pose/info]
[ign gazebo-3] [Msg] [EntityComponentManager.cc:1623] Updated state thread iterators: 2 threads processed around 41 entities each.
[ign gazebo-3] [Msg] [SimulationRunner.cc:523] Exiting postupdate worker thread (1)
[ign gazebo-3] [Msg] [SimulationRunner.cc:523] Exiting postupdate worker thread (0)
[ign gazebo-3] [Msg] [SimulationRunner.cc:523] Exiting postupdate worker thread (2)
[ign gazebo-3] [Msg] [SimulationRunner.cc:496] Creating PostUpdate worker threads: 0
[ign gazebo-3] [Msg] [SimulationRunner.cc:507] Creating postupdate worker thread (0)
[ign gazebo-3] [Msg] [SimulationRunner.cc:507] Creating postupdate worker thread (1)
[ign gazebo-3] [Msg] [SimulationRunner.cc:507] Creating postupdate worker thread (2)
[ign gazebo-3] [Msg] [SimulationRunner.cc:507] Creating postupdate worker thread (3)
[ign gazebo-3] [Msg] [SimulationRunner.cc:507] Creating postupdate worker thread (4)
[ign gazebo-3] [Msg] [RenderUtil.cc:2591] Create scene [scene]
[ign gazebo-3] [Msg] [Sensors.cc:610] Initialization needed
```

Key Performance Metrics



Mean Average Precision

Comprehensive mAP across all classes



F1-Score Balance

Precision-recall optimization

Custom Military Target Model



YOLOv8 Model



26,315 labeled images



12 military-relevant classes



Highly specialized for defense applications

Military-Relevant Classes



camouflage_soldier



weapon

military_tank

military_vehicle



chemical_vehicle



radar_system



Training Approach

The model was specifically trained on a diverse dataset of military-relevant classes to ensure high detection accuracy for defense applications. The dataset was carefully curated to provide balanced representation across all 12 classes.

Object Detection Model Performance

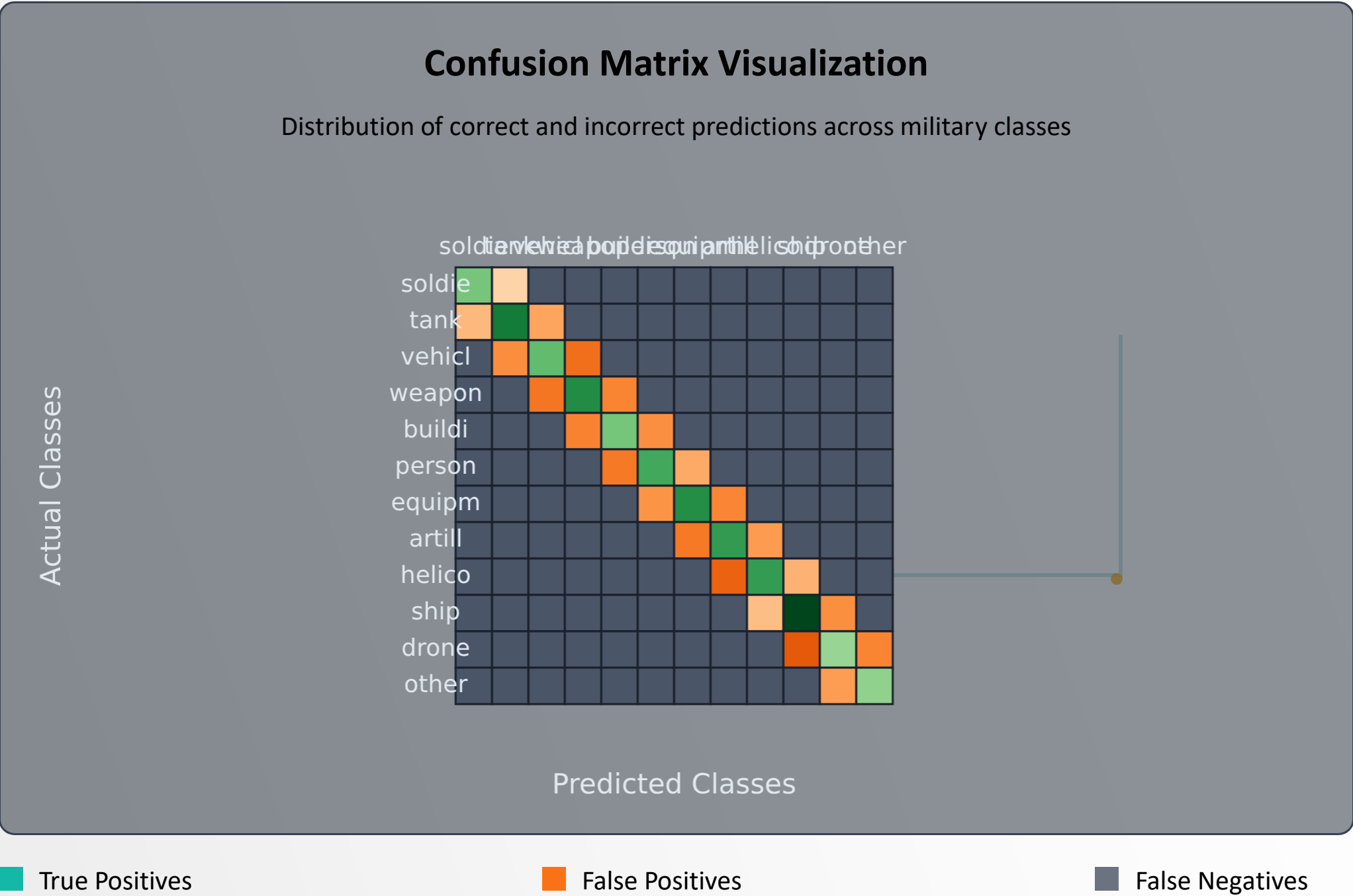
Confusion Matrix Analysis

The confusion matrix visualizes predictions across 12 military classes, highlighting true positives, true negatives, and errors.

- ✓ High accuracy for critical classes like camouflage_soldier and military_tank
- ⚠ Higher false positive rates for cluttered background classes

Key Insights

- ➔ Model demonstrates strong discriminative capability
- ➔ Training on 26,315 images ensured robust feature learning



Detection Accuracy Metrics

Training Results Convergence

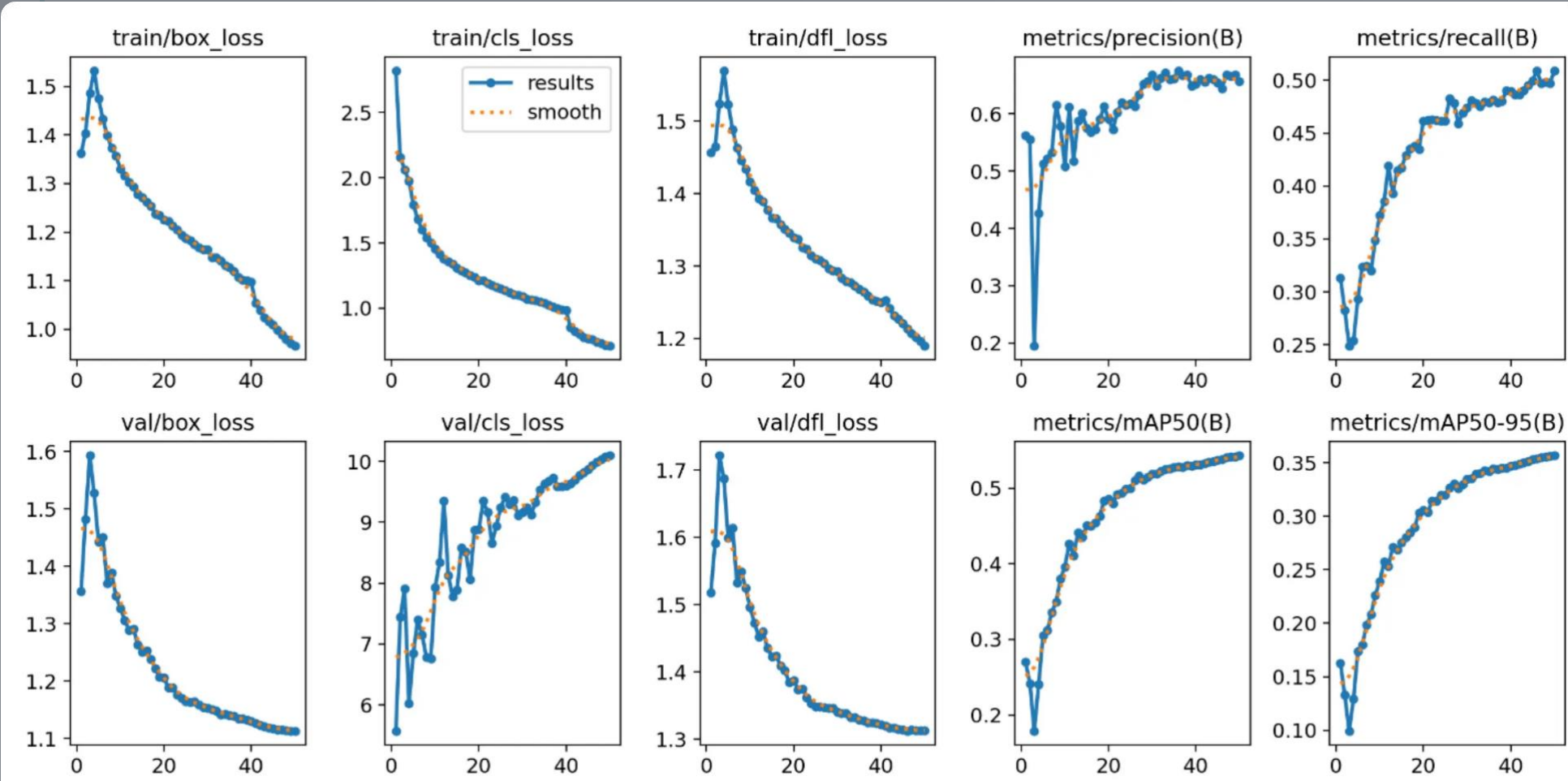


Figure 34. Training Result

Figure 34: Model training convergence

★ Mean Average Precision

The model achieved strong mAP performance across all 12 military-relevant classes, indicating robust detection capability.

⚙️ Critical Class Accuracy

Camouflage Soldier **High**



Military Tank **High**



Other Critical Classes

Medium-High



🗄️ Training Dataset

26,315 labeled images across 12 military-relevant classes

camouflage_soldier 0.36



ge_soldier 0.39



weapon 0.28



military_tank 0.85



military_tank 0.86



military_tank 0.93



Real time test of the YOLOv8 Model

E

F

Conclusion



Modular Defense System Design and
and simulation using ROS



YOLOv8 Integration in ROS 2



Omni-directional Control Validation



Multi-Sensor Integration

These achievements demonstrate the successful development of a **ROS 2-native, modular, and scalable** automated defense automated defense system for mobile robots.

E

F

Future Work and Development



Physical Prototype Deployment



Expanded Training Dataset



Advanced Navigation Algorithms



Physical Actuation Module

YOLOv8 Integration in ROS 2



Thank You

Ramzi Abidi

